

Game-theoretic Foundations for the Strategic Use of Honeypots in Network Security

Christopher Kiekintveld
Viliam Lisý
Radek Píbil

Abstract An important element in the mathematical and scientific foundations for security is modeling the strategic use of deception and information manipulation. We argue that game theory provides an important theoretical framework for reasoning about information manipulation in adversarial settings, including deception and randomization strategies. In addition, game theory has practical uses in determining optimal strategies for randomized patrolling and resource allocation. We discuss three game-theoretic models that capture aspects of how honeypots can be used in network security. Honeypots are fake hosts introduced into a network to gather information about attackers and to distract them from real targets. They are a limited resource, so there are important strategic questions about how to deploy them to the greatest effect, which is fundamentally about deceiving attackers into choosing fake targets instead of real ones to attack. We describe several game models that address strategies for deploying honeypots, including a basic honeypot selection game, an extension of this game that allows additional probing actions by the attacker, and finally a version in which attacker strategies are represented using attack graphs. We conclude with a discussion of the strengths and limitations of game theory in the context of network security.

Christopher Kiekintveld
Computer Science Department, University of Texas at El Paso, USA, e-mail: cdkiekintveld@utep.edu

Viliam Lisý
Agent Technology Center, Dept. of Computer Science and Engineering, Faculty of Electrical Engineering, Czech Technical University in Prague, e-mail: viliam.lisy@agents.fel.cvut.cz

Radek Píbil
Agent Technology Center, Dept. of Computer Science and Engineering, Faculty of Electrical Engineering, Czech Technical University in Prague, e-mail: radek.pibil@agents.fel.cvut.cz

1 Introduction

We are increasingly reliant on an infrastructure of networked computer systems to support a wide range of critical functions in both civilian and military domains. A major challenge we currently face is securing these networks against malicious attackers ranging from individuals to criminal organizations and nation states. The diverse and constantly evolving nature of the threat and the tactics used by attackers demand that we develop a similarly diverse and adaptive array of countermeasures for securing networks. In addition, we need to develop the mathematical foundations and scientific understanding of security necessary to accurately model the threat and to effectively deploy these countermeasures. The science of security focuses not on identifying specific vulnerabilities or attack/defense techniques, but rather on studying more fundamental principles of security.

A important feature of many security problems is that they involve aspects of deception, information hiding, or information manipulation. For example, the first stage in executing an attack on a network is typically reconnaissance—scanning a network and using fingerprinting techniques to gain information about the connectivity and configuration of the network infrastructure and individual hosts. Intrusion detection also involves manipulating information on both sides; attackers try to disguise their activities to make it difficult to distinguish between attacks and legitimate activity, while network operators try to detect attacks and learn as much as possible about the behavior of attackers. Some code injection vulnerabilities can be mitigated by randomizing code and instruction sets [10], which can be viewed as another use of information hiding. We will focus here on examples that involve deploying honeypots (i.e., fake computer systems) to attract the attention of attackers to gain information about the attackers and waste their resources.

We argue that game theory provides a valuable theoretical foundation as well as specific models and algorithms that can be applied to many problems in network security and cyber warfare. Game theory studies decision-making problems with multiple interacting decision makers (also called agents or players), including adversarial situations where two or more agents have opposing goals. Security problems almost always involve interactions between multiple agents with differing goals (e.g., an attacker and a network administrator), so game theory is a natural framework to apply to these situations. Game theory is also particularly well-suited to reasoning systematically about uncertainty, strategic randomization, and information manipulation in the context of adversarial agents. The notion of mixed strategies that allow players to randomize their actions is central to game theory, and many solution concepts can provide guidance on exactly *when and how* to randomize in cases where a uniform random strategy is suboptimal. Game theory also emphasizes the beliefs that players hold, both about the state of the world and their mental models of how other players will act. Reasoning about such beliefs is an important aspect of understanding deception at the deepest level.

Here we present several game-theoretic models that focus on the strategic use of honeypots in network security. These models involve deception in the sense that honeypots are fundamentally about deceiving attackers into attacking fake targets, and they also involve deception in the sense that randomization is necessary in the optimal strategies. Honeypots serve several purposes in protecting networks, including early detection of attacks, collecting detailed information about attacker behaviors and capabilities, distracting attackers and wasting resources, and making it more difficult for attackers to gain an accurate understanding of the configuration of a network. However, they are limited resources, and there are strategic decisions about how to deploy them to maximize the benefit in terms of improve security. Our models address these resource allocation questions for honeypots at varying levels of abstraction.

We begin by briefly introducing background material on game theory, and then on two classes of games that are closely related to network security problems: security games [12, 28] and deception games [26]. We then describe our basic model, the honeypot selection game, followed by our extension of this model that includes additional probing actions that attackers can use to detect whether a particular machine is a honeypot or not. The next model considers a richer representation of attacker behavior in the form of attack graphs; we consider the problem of optimizing honeypot deployments to interdict possible attack plans in this representation. We conclude by discussing issues related to using game theory for network security, including strengths and weaknesses of the approach and opportunities for future work.

2 Background

We begin by introducing some basic background material on game theory; for a more thorough introduction, see [25]. We then discuss two classes of games from the literature that are particularly relevant to cyber warfare.

Security Game

2 players

2 targets

1 defender resource



Target 1 Target 2



Target 1

Target 2

	Target 1	Target 2
Target 1	1, -1	-2, 2
Target 2	-1, 1	2, -1

2.1 Game Theory

Fig. 1 A an example of a game in normal form.

Game theory is used in many disciplines as a framework for modeling situations with more than one decision maker (player) [16, 25]. One of the important features of games is that there is no straightforward way to define

the optimal action for a given player, because the best action depends on the behavior of the other players. Advances in game theory have led to many different ways of modeling games with different features, and solution concepts and algorithms for reasoning about strategy choices and outcomes.

The elements of a game model include:

1. The **players** who participate in the game
2. The **strategies** available to each player
3. The **payoffs** to each player for each combination of strategy choices (outcome)
4. (optional) Information about the sequences of moves made by the players
5. (optional) A definition of what information players have when making moves

One way of representing games is the *normal form*, often depicted using a matrix such as the one shown in Figure 1. This example is a simple security game with two players, an attacker and a defender. The defender's strategies are represented by the rows: defend target 1 (top), or defend target 2 (bottom). The attacker's strategies are the columns: attack target 1 (left), or attack target 2 (right). Each cell in the matrix gives the payoffs for one of the four possible outcomes. For example, the upper left cell is the case where both the attacker and defender choose target 1, resulting in a payoff of 1 for the defender and -1 for the attacker.

Another important concept in game theory is *mixed strategies*, which allow players to randomize their play by selecting according to a probability distribution over their pure strategies. For example, a defender using a mixed strategy in the example game could choose target 1 60% of the time and target 2 40% of the time. It is clear that randomization is important in many games as a way to keep an opponent from predicting what strategy will be played. In the well-known game of "Rock, Paper, and Scissors," any predictable strategy can be easily defeated, and players must be able to select different actions to be unpredictable; mixed strategies capture this type of behavior in games. Mixed strategies can also be interpreted as a form of deception or information hiding, where the information that is hidden is what the player's strategy choice will be.

Games can be analyzed in many ways, but typically the goal is to predict what behaviors are likely to be observed when the game is played, or to identify a strategy for a player to play that will result in a high payoff (of course, selecting a good strategy may also involve making predictions about what the other players will do). Methods for analyzing games are known as solution concepts, including the well-known concept of Nash equilibrium. A *Nash equilibrium* is a profile of strategy choices for all players such that every player is playing optimally, given strategies of the other players. This concept requires that every player correctly predicts the strategies of the other players, and plays an optimal strategy in response (known as a best response). In a Nash equilibrium, every player could announce their strategy choice, and

no player would have an incentive to change to a different strategy. Nash’s seminal result showed that such equilibria always exist in finite games as long as players are allowed to play mixed strategies that allow randomization [15].

In addition to the basic normal form representation, game theory includes a large number of richer representations that can be used to describe sequential interactions between players, the structure of the (imperfect) information that players have available when making decisions, and other complexities of a particular situation. There are also many different solution concepts beyond Nash equilibrium. Some are extensions of Nash equilibrium to more complex game representations, and others are refinements that place additional restrictions on the behaviors of the players. Still others use different types of reasoning to find solutions, or are motivated by descriptive goals such as predicting the play of human players in games. A thorough review is beyond the scope of this article, but we will discuss a few specific models and concepts that are most closely related to security.

2.2 Security Games

Security problems often involve making complex decisions in adversarial settings where the interests of attackers and defenders are directly opposed. A recent line of research has adopted game theory as a framework for randomizing security deployments and schedules in homeland security and infrastructure protection domains. Security games [12, 28] provide a general model for resource allocation decisions in adversarial settings where attackers can observe and exploit patterns in the deployments of resources. This model has been successfully used to randomize the deployment of security resources in airports [19, 20], to create randomized schedules for the Federal Air Marshals [30], and to randomized patrolling strategies for the United States Coast Guard [24].

The example in Figure 1 is a very simple example of a security game. More generally, a security game specifies a set of possible targets (e.g., flights, train stations, inbound roads to an airports). The attacker picks one of these targets, while the defender uses resources to protect the targets. However, the defender has limited resources and cannot protect all of the targets, and therefore must select only a subset to protect. The payoffs in the game specify four values for each target, two for the defender and two for the attacker. The pairs of values correspond to whether or not there is a defender assigned to the target or not. The defender receives a higher value for the case where the target is protected than when it is unprotected, while the attacker receives a higher value if the target is unprotected. A special class of games, *zero-sum games*, is often used to model purely adversarial settings in which the payoffs of the players are exactly opposite (i.e., one player wins exactly the amount that the other player loses). Security games have a similar structure in that

payoffs move in the opposite direction, but they allow for more variation in the payoffs than zero-sum games.

It is common in security games to model the attacker as having the capability to observe and learn about the defender strategy before deciding on an attack. This is a realistic assumption for many domains, since attackers typically use surveillance and other information gathering actions during the course of planning an attack, so they are likely to have significant knowledge of the defenses protecting a target before deciding to attack. The Stackelberg security game model incorporates this assumption by assuming that the defender moves first and commits to playing any mixed (randomized) strategy. The attacker observes exactly what this mixed strategy is, and then calculates an optimal target to attack using this knowledge. This is the worst case in a sense, since it assumes that the attacker gains perfect information about the defense strategy. Other models have also been explored in the literature, including simultaneous moves and cases with various types of imperfect observations for the attacker.

The key question in security games is to decide how the defender should allocate limited protective resources to maximize protection against an intelligent attacker who reacts to the security policy. The optimal deterministic strategy is simply to protect as many of the most valuable targets as possible. However, a randomized strategy can use deterrence to improve the overall level of protection by spreading the resources out over a larger set of targets. Since the attacker does not know exactly which targets will be protected (only the probabilities), the attacker has some risk of being unsuccessful when attacking a larger set of targets. Essentially, the defender is able to gain an advantage by concealing information about the defense strategy using randomization. However, a strategy that optimally conceals information is the uniform random policy that uses the same probability to protect every target. This is also not optimal for the defender; the defender should weight the probabilities so that the more important targets are protected with higher probability. The crux of the game-theoretic solution for a security game is to find a weighted randomized strategy for allocating the security resources that optimally balances between allocating more resources to higher-valued targets and hiding information about the policy using randomization to increase risk for the attacker.

Strategic decision-making in cyber warfare poses many of the same fundamental challenges as protecting physical infrastructure: limited resources, the need to manipulate the information an attacker can gain through surveillance, and the need to learn about an attacker's strategy. We believe that game theory can be used to address many of these challenges in cyber domains, building on the success of security games. One promising area is developing better strategies for using honeypots and other deception strategies in network security. We describe some of our initial research on game-theoretic models for deploying honeypots to illustrate this approach below.

2.3 Deception Games

Deception has been studied in many forms in the game theory literature. It often takes the form of using randomized strategies to hide information from an opponent, as in the security games described above. Another example is found in game theory models of poker, which naturally generate bluffing behaviors as part of randomized Nash equilibrium strategies. In these strategies, it is optimal for players to sometimes make bets with poor cards so that it is more difficult for an opponent to infer information about the cards a player is holding based on their bidding strategy.

Another model that has received some attention in the game theory literature models deception more directly using actions intended to confuse an opponent. A formal model of *deception games* was first introduced in [26]. One player is given a vector of three random numbers drawn from the uniform distribution on unit interval. This player is allowed to modify one of the three numbers to any other number from the same unit interval. The modified vector of numbers is shown to the second player. The second player chooses one of the three numbers, and receives the original value of the number at that position as a reward (i.e., the value before the modification made by player 1). So, the second player prefers to choose higher values, but the first player can try to deceive the second player by making one of the small values in the vector appear large. The open question stated in the original paper is whether there is a better strategy for the second player than randomly choosing one of the numbers. This question was answered in [13] proving that if the changer plays optimally, the second player cannot play better than random. Several similar questions about various modifications of the model were published in the next years, but the results generally apply only to the specific game formulations and they do not present the complete strategies to play the game. Our model for selecting honeypots builds on this deception game model, though it is not identical in the details.

3 Honeypots

A honeypot is a computer system deployed in a computer network specifically to attract the attention of possible attackers. They do not store valuable information, and are designed to log as much information about system behavior as possible for analysis after an event. Honeypots can help to increase system security in several ways [27]: (1) The presence of honeypots can waste the attacker's time and resources. The effort an attacker spends to compromise the honeypot and learn that it does not contain any useful information directly takes away time and resources that could be used to compromise valuable machines. (2) Honeypots serve an intrusion detection role, and can provide an "early warning" system for administrators, providing more time

to react to attacks in progress. A particularly important case of this is detecting “zero day” attacks and previously unknown vulnerabilities. (3) Once an attacker compromises a honeypot, the network administrator can analyze all of the attacker’s actions in great detail, and use the information obtained to better protect the network. For example, security gaps used in an attack can be patched, and new attack signatures added to antivirus or intrusion detection systems.

We consider the use of honeypots from an adversarial perspective, where network administrators and attackers both reason strategically. In general, a network administrator will try to maximize the probability that an attacker chooses to attack a honeypot rather than a real service. However, attackers are increasingly aware of the possibility of honeypots, and actively take steps to identify and avoid them. For example, once they gain access to a system, they can use multitude of methods to probe the system and rule out the possibility that they are in a honeypot before they continue with their attack (e.g., [5]). To be effective against more sophisticated attackers, honeypots must be sufficiently disguised that they are not obvious (i.e., they cannot simply present the most vulnerable possible target).

Honeypots can be classified into low interaction and high interaction variants. A *low interaction* honeypot is relatively simple, and therefore it can be added to the network at low cost [4], but even simple probing by the attacker will reveal it is not a real system. A *high interaction* honeypot is much more expensive to create and maintain. For example, to increase believability realistic user activity and network traffic may need to be simulated. High interaction honeypots in particular are a limited resource and it is important to optimize how they are deployed.

4 The Honeypot Selection Game

In Píbil et al. [18] we introduced the *Honeypot Selection Game* to model situations where an attacker decides which machines or services in a network to attack, and a defender decides what types of honeypots to add to the network. One of the important features of real-world networks is that they have many different types of machines with different configurations (available services, hardware, etc.). Some categories of machines are more important than others, both to the owner of the network and as targets for the attacker. For example, a database server containing valuable military information may have a much higher value than a standard laptop. To model this we classify machines into categories of *importance* that are assigned values to represent the gain/loss associated with a successful attack. One of the decisions that the defender makes when deploying honeypots on a diverse network is how to disguise the honeypots – in other words, which category of machine should each honeypot be designed to look like?

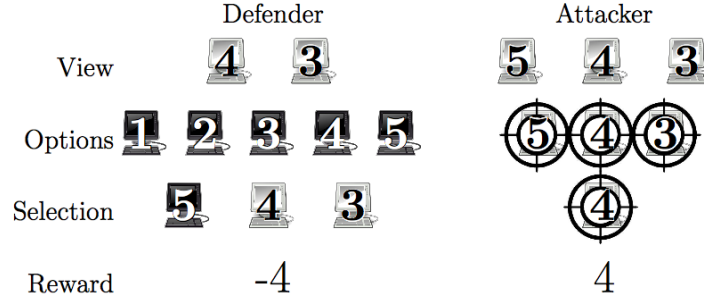


Fig. 2 Example run of the Honeypot Selection Game. The defender selects one honeypot to add to a network with two targets. The attacker attacks a single target and cannot distinguish honeypots from real hosts.

We represent a *configuration* of the network using a vector of values representing the apparent importance of each machine. The defender knows the values of each of the real machines in the network, and extends the vector by adding honeypots. For each honeypot, the defender selects the value of the machine that will be observed by the attacker (e.g., by configuring the honeypot to emulate machines of that category). We assume that both players have knowledge of the typical configurations of the network, so the players can calculate the probability that the configuration is the actual configuration of the network. We assume that the defender adds a fixed number of honeypots to the network, and that the attacker knows the number of honeypots. This is a worst case assumption about the attacker, and the model could be generalized to allow for imperfect information about the number of honeypots, though it makes the problem more difficult to solve.

Consider the example shown in the Figure 2 with two machines that have importance values 4 and 3. The administrator has one honeypot and needs to decide how to assign it a value. He could assign it a value of 5 to make it appear very attractive (e.g., by making it appear to contain valuable data and exposing obvious vulnerabilities). The attacker observes the unordered vector of values by doing a scan of the network, including the value of the honeypot: (5,4,3). A naïve attacker might attack the machine with the highest value (5), therefore attacking the honeypot. However, a sophisticated attacker might reason that this is “too good to be true” and choose instead to attack the next best machine, with a value of 4. If the attacker chooses a real machine to attack, he obtains a reward and the network administrator is penalized. If the attacker chooses to attack a honeypot, he does not obtain any reward and possibly is penalized for disclosing his attack strategy.

From this example, we can see that assigning all honeypots a maximal value may not be the optimal strategy. We search for a solution in the form of a Nash equilibrium, which is a pair of mixed strategies such the neither player

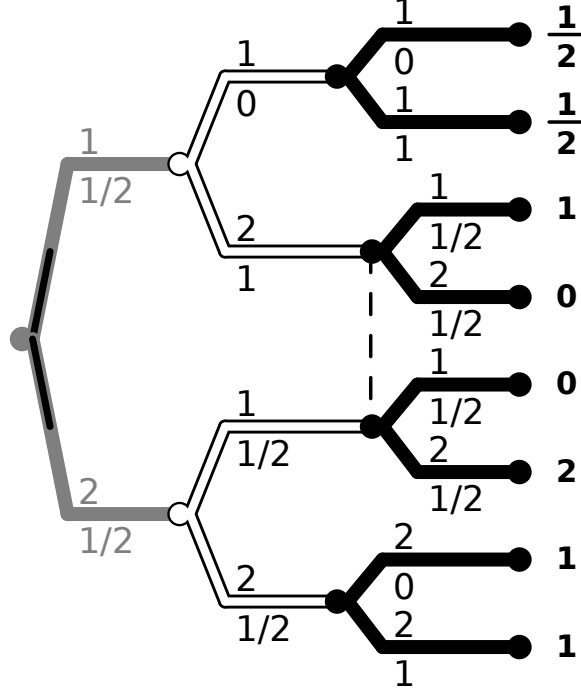


Fig. 3 The game tree of a Honeypot Selection Game rendered by Gambit with one real service, one honeypot and a domain 1, 2. Light gray edges are random choices, white edges are defender's actions and black edges the attacker's actions. Services corresponding to actions are above the branches, while probabilities are under them.

can benefit by changing to a different strategy. To find a Nash equilibrium we can formulate this as an optimization problem (due to the specific structure of the game), and use linear programming to find a solution; for details of the formulation and optimizations that speed up the search see [18]. For very simple games we can also visualize and solve the game models using Gambit, a well known toolkit for game theory. An example of a game tree using gambit is shown in Figure 4.

The typical form of a solution can be seen in Figure 4(a) (left), which shows the probability that the defender will use a honeypot with each different value from 1 to 4 for a scenario with uniform random values for the machines in the network (full details of the experiments are presented in [18]). In general, higher values are used more frequently but the optimal probability values depend on the details of a specific problem. If we assume that all network configurations are equally probable then the optimal strategy for the defender in the situation from Figure 2 is to add the honeypot with value 4. The strategy for the attacker is then to select a target randomly with uniform probability.

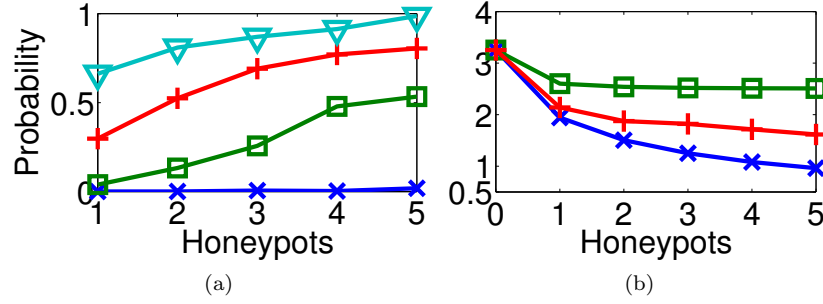


Fig. 4 (a) An example of the structure of a typical solution, where each line represents a honeypot value from 1–4, with higher lines corresponding to higher values. The number of honeypots allowed varies along the x axis. (b) The defender's expected utility against three different types of attackers as the number of honeypots allowed increases (Cross: Nash equilibrium attacker, Plus: random attacker, Box: Maximum attacker).

In Figure 4(b) we present the results for the defender's expected value against three different types of attackers as the number of honeypots increases. The box line is an attacker who always chooses the maximum value (i.e., an optimistic attacker who ignores the possibility of honeypots). The plus line is an attacker who chooses a target at random, without respect to the value of the target. The cross line is an attacker who accounts for honeypots and plays according to the Nash equilibrium of the game.

5 Honeypot Selection with Attacker Probes

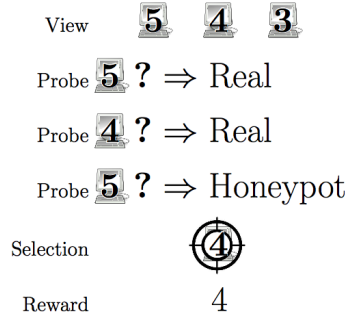


Fig. 5 Demonstration of the attacker's actions in Honeypot Selection Game with Probes

There are many opportunities for interesting extensions to this basic model. One that we have explored in detail allows attackers to execute *probe* actions to further analyze the machines on a network to determine whether or not they are honeypots. This is intended to model the real behaviors of attacker in networks who use various strategies such as traffic analysis to identify likely honeypots [5].

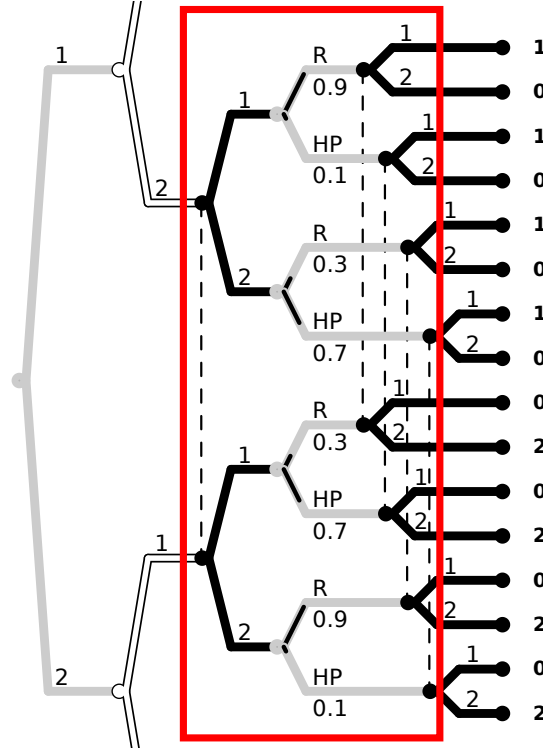


Fig. 6 An example of part the Gambit game tree for an example of the Honeypot Selection Game extended with probing actions. The area in the box represents a probing action for the attacker and the observation revealed by nature.

To capture this situation in the formal game theoretic model, we introduce additional probe actions for the attacker. Prior to executing the attack on the selected target, the attacker executes a limited number of probing actions on the hosts of his choice. The result of the probe is an observation of whether the probed host is real, or a honeypot. This observation is closely, but not perfectly, correlated with the real nature of the host. The probing methods are typically not perfect and the defender can even disguise real servers as honeypots in order to deter the attackers [21]. The probes are executed sequentially and the attacker can use the results of the earlier probes to decide which host to probe next. After the probing phase, the attacker has

to combine the information acquired by the probing and his believe about the strategy deployed by the defender to pick one target to attack.

An example showing part of the game tree with probing actions for the attacker is shown in Figure 5. Comparing this figure with the game tree for the basic honeypot selection game we can see that the game is much larger, due to the additional levels of actions for the attacker when selecting probes. In addition, the information sets are significantly more complex, since the attacker can gain different types of information during the game. While we can still formulate and solve these games using linear programming, the size of the game poses additional computational challenges that increase with the number of probing actions allowed.

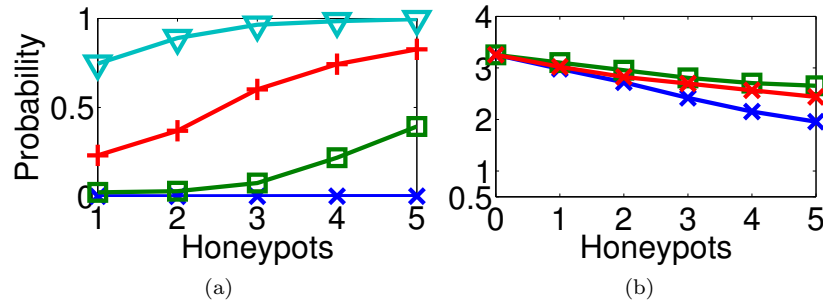


Fig. 7 (a) An example of the structure of a typical solution for the version of the game with probes, where each line represents a honeypot value from 1–4, with higher lines corresponding to higher values. The number of honeypots allowed varies along the x axis. (b) The defender's expected utility against three different types of attackers as the number of honeypots allowed increases (Cross: Nash equilibrium attacker, Plus: random attacker, Box: Maximum attacker).

For the game with probes with also compute the optimal solution in the form of a Nash equilibrium. Similar results to those in the previous section are presented in Figure 7(b). The strategy for the defender is shown in the plot on the left. When the defender takes into account the probing of the attacker, the optimal strategy uses the most important honeypots more often than in the case without probes. This is mainly because the rational attacker directs the probes to the most promising targets and probing a honeypot typically has higher error probability than probing a real machine. The expected payoff for the defender against the same three types of attackers (maximum, random, and Nash equilibrium) is shown on the right.

6 Honeypot Strategies for Games on Attack Graphs

The models discussed above use a relatively simple representation of the attacker strategies, with the attackers choosing only which machines to attack. However, richer models of attacker behavior are highly desirable, particularly when viewing honeypots as a mechanism for learning about the plans or behaviors of attackers. One useful modeling paradigm that captures a broader range of attacker behavior is attack graphs (sometimes restricted to attack trees). Attack graphs represent all the known attack vectors that can be used to compromise the security of a system. Several different types of attack graphs have been proposed in the literature, but they share the basic idea of representing sequential attack plans in which attackers use various actions to gain privileges on certain machines in a network, which in turn may allow new attack actions.

One advantage of adopting the attack graph formalism is that there are several existing tools that can automatically analyze a computer network to generate attack graphs based on known vulnerabilities. For example, the network scanning tools Nessus¹ and OpenVAS² scan the network (or individual hosts) for open ports, installed software versions and similar characteristics, and based on large databases of known vulnerabilities identify the ones that are present in the network. Vulnerability databases can also provide additional structured data that is useful for creating models, such as assessing the probability of success or the expected cost of exploiting specific vulnerabilities. The vulnerabilities found by network scanners can then be merged to create attack graphs using rule-based tools, such as MulVAL[17] or NetSPA [8]. A very simple example of an attack graph is shown in Figure 6.

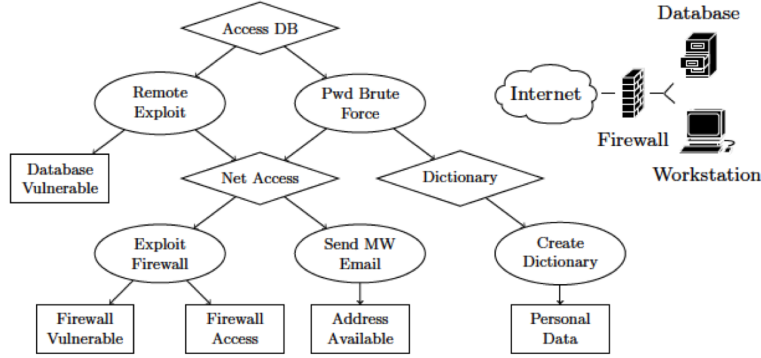


Fig. 8 An example of a simple attack graph.

¹ <http://www.nessus.org> ²<http://www.openvas.org>

Existing work mainly uses attack graphs to identify the optimal subsets of vulnerabilities to fix on the network, in order to perfectly secure the network against the known attacks or to optimize some aggregate measure of the overall security of the network. In game-theoretic models, attack graphs can be used to model complex behavior of the attackers. Based on analysis of the attacker behaviors we can consider the optimal strategies for the defender, which can include various policies or configuration changes to the network, such as adding honeypots to the network or modifying firewall rules.

If both the administrator and the attacker have perfect knowledge about the network and the security measures deployed on the network, the defender can optimize the security measures with respect to the expected reaction of the attacker. For example, the defender can try to identify the best possible set of rules for the firewalls in the network. For each such set, the defender can (1) evaluate the restrictions to the usability of the network, (2) generate the attack graph representing all possible attacks after the new rules are applied and based on that, (3) compute the optimal attack with respect to the time or skill level required by the attacker. Among all the evaluated options, the defender can choose the one that achieved the desired tradeoff between security and usability.

In the honeypot selection game, we can use the more detailed modeling of the attacker's behavior allowed by the attack graphs to better determine the importance of a host in the network. Instead of an abstract number representing the importance of a host, we can use the attack graph to evaluate the role of the host in the potential attacks. For example, we can estimate how costly the attacks using a specific host are in comparison to the attacks that do not use this host. We can estimate how many options for attack continuation compromising each of the hosts will open. If analysis of the attack graph identifies a key node where an attacker might try to exploit a particular vulnerability, adding several honeypots to the network that appear to replicate this key node can result in a high probability that the attacker chooses to attack one of the honeypots before attacking the real machine.

We assume that the defender can choose a subset of honeypots from a fixed set of alternatives. There are endless number of possible configurations of the honeypots and their placement into the network; hence, in our initial investigation we restrict the defender to use only honeypots that completely copy the configuration and connectivity of an existing host on the network. This restriction has several advantages. First, copying an existing machine and obfuscating the useful data on the machine can be done almost automatically with a substantially lower costs than configuring a brand new host. Second, it can create much more believable honeypots that will make it hard for the attacker to distinguish them from the production systems. Third, if the attacker uses some unknown exploit on the honeypot, we know that the same exploit is available on one of the production machines. We can check these machines to determine if they were compromised in the same way, and try to remove the vulnerability and/or update the attack graph to include

it in further analysis. Fourth, we do not introduce any fundamentally new attack vectors to penetrate the network. A honeypot can take the place of a production machine in the attack plan, which is a the desired effect, but we do not open any new opportunities for the attacker. Hence, it is virtually impossible that we are reducing the security of the network. Moreover, the attack graph created for the original network perfectly represents the possible attack on the modified network, if we abstract the exact machine on which the individual attack actions are performed. As a result, the effect of adding each subset of honeypots to the network is only modifying the probabilities that certain attack actions use a honeypot and not the attack graph itself.

Copying each of the hosts has different initial and maintenance costs for the defender. The attacker has costs, for example, for the time he spends attacking and the possibility of disclosing a zero-day exploit to the defender. The players generally do not care about these costs of their opponents. Furthermore, the defender often values his sensitive data more than the attacker trying to steal them. Therefore, we model the game as a non-zero-sum game and we use the Stackelberg equilibrium as the solution concept for the game.

In a Stackelberg game, we assume that the attacker knows the action selected by the defender. In this case, he knows what hosts on the network have been copied as honeypots, but we assume that he does not know which particular instance of certain type of host is the real one and which is the honeypot. Based on that, he can derive the probability that each of his actions will interact with a honeypot and use this information in computing the optimal attack to perform on the network. The optimal set of honeypots for the defender to add to the network is the one that motivates the attacker to select the attack that is least harmful to the defender. The attacker will either pick less valuable targets or have a high probability of being discovered.

The main challenge that this model poses is the computational complexity of computing optimal solutions. We have proposed a method for computing the optimal attack by transforming the problem into standard problems previously solved in artificial intelligence [14] as well as a more efficient problem specific method [6]. Both of these methods can be extended to take into account the probability of interacting with a honeypot in computing optimal attack strategies. While the optimal solvers are still only able to solve small networks, the game theoretic models build on top of these solvers can use the fact that the same problem with mild modifications is solved repetitively to provide many improvement, such as reusing partial results and deriving strong heuristics.

Either way, solving the optimal attack problem is NP-hard if both the cost and probability of success of attack actions are considered. Therefore, it may be very interesting to integrate approximate solvers with error guarantees, such as [2], to the game theoretic models. However, this requires investigating new solution concepts which would be able to deal with this uncertainty.

7 Discussion

We now discuss some of our observations related to deploying game-theoretic models and solution methods for network security and cyber warfare. We first highlight some of the more important strengths of the approach:

- Game theory provides a strong theoretical/mathematical foundation for modeling adversarial decision, including security
- The models assume intelligent adversaries that adapt to defensive strategies
- Game theory offers models of deception and strategic randomization that can be used to systematically identify good strategies for manipulating information
- There are many existing models and algorithms that can be leveraged for analysis
- Modeling player's beliefs play a central role, including recursive modeling of the beliefs and strategies of the other players

There are also a number of challenges in using game theory for security, including:

- Network security problems are often very complex with combinatorial action/belief spaces, resulting in scalability problems for analyzing the games
- Large models contain many parameters, and these may be difficult to estimate efficiently using available data
- Game theory models are best suited to maximizing expected results, rather than providing guaranteed performance
- Some types of game models and solution concepts make implausible assumptions including perfect rationality and perfect knowledge

We begin by highlighting some of the features of game theory that make it a promising direction for establishing a stronger scientific approach for security. The main feature of game theory is that it explicitly models all of the agents in a scenario, including their available actions and the values that they assign to the various outcomes. This is a rich modeling framework, and it is able capture how players will *adapt and respond* to the actions of other players. In contrast, many other security decision making frameworks really on a *fixed* notion of the threat or risk, such as a probability that an attacker will attempt a particular type of attack. Such models are more limited than game-theoretic models because they do not offer predictions of how the threat will change as the defensive strategy changes. A model with a fixed threat is roughly equivalent to a myopic best response calculation in game theory (i.e., given my current prediction about the threat, what is the optimal defensive strategy?). Game theoretic models offer the possibility of thinking further ahead to anticipate responses by an opponent, and even calculating the equilibria that are fixed points of the best-response dynamics and other evolutionary processes.

There are challenges in constructing a useful game-theoretic model that captures the important features of a problem at the right level of abstraction, and it should be emphasized that game theory may not be useful for all security problems. In some cases the solution to a problem may be trivial or immediately obvious due to the symmetry or other structure in the problem. For example, if one wants to make it difficult to discover the IP address of a machine and all IP addresses are functionally identical, assigning the addresses using a uniform random policy is trivially the best solution.

Game theory is also not very useful in situations where a guaranteed solution is desirable (e.g., formally proving that systems will never enter undesirable states, or that a cryptographic protocol is secure). However, we argue that often there is no such thing as 100% security, or it is cost prohibitive relative to the value of the resource being protected. For these situations we must accept results based on expected outcomes, accepting an appropriate level of risk. Game theory is best suited to models problems where there are complex interactions between different strategies, and it is necessary to make choices that maximize the *expected* value of a specific course of action. For these cases the solutions to the game will balance risk and cost by selecting strategies with high expected values.

Constructing a useful game theoretic model can also be difficult due to the complexity that arises in many security domains. Computer networks are very complex in that they comprise large numbers of different types of interconnected devices, running different software and services, and with different owners. These machines are constantly communicating in varying patterns using complex protocols. Modeling these systems can easily lead to combinatorial spaces, such as selecting subsets of the hosts in a network, or generating all possible routes/paths through a network, or modeling different combinations of software and services that could be present on a host. When decisions are being made for these systems, it is easy to generate models that are intractable due to the size of the game. A model may be intractable to solve due to limited scalability of the solution algorithms, and the models may also be problematic to specify due to the number of parameters.

There is reason for optimism in developing game theory models for very complex domains. Much of the progress in security games has been driven by advances in algorithms for scaling to very large problem instances, including some where the strategy spaces involve large networks [9, 29]. In addition, there has been dramatic progress on algorithms for solving poker and related games, with new algorithms able to solve games that are several orders of magnitude larger than the best algorithms from just a few years ago [22]. These scalable solution algorithms are based on key ideas such as iterative solution methods that avoid enumerating the full strategy space [1], and automated abstraction methods that can simplify large games to more tractable versions [23]. We believe that applying such techniques to network security games could lead to similar advances and allow for many new applications of game theory for network security.

A related challenge for modeling cyber security domains is that these domains are very dynamic, and the details of the decision problems are constantly changing. Computer networks are not static entities; hosts may join or leave the network, network infrastructure and routing may change, the configurations of individual hosts are updated as software and services are added, removed, or updated. The nature of the threat also changes as new vulnerabilities are discovered or new attackers with different goals and capabilities attempt to compromise a network in different ways. These changes mean that the relevant models for decision making must also be updated on a continual basis so that they are still relevant.

The dynamism of cyber security domains presents opportunities for using learning in combination with game theory. Many previous security games models have focused on terrorism, and there is very limited data available to construct and validate the models since there are very few examples of terrorist attacks. In cyber security, interactions between defenders and attackers are much more frequent, with results in a much larger amount of data that can be used to specify models and to improve them over time. The field of empirical game theory [11] has introduced methods for constructing game-theoretic models using data. There is also a large body of work on multi-agent learning [7] that is relevant for situations where multiple agents interact frequently and can learn about the behavior of other agents. Both of these are promising directions for future research on making dynamic games applicable to cyber security.

Another issue that game theory raises that is of particular interest for cyber security is accurately modeling the behavior of agents that are not perfectly rational. Research on behavioral game theory focuses on developing descriptive models that accurately predict human behavior in games [3]. The solution concepts studied in behavioral game theory differ from Nash equilibrium and other traditional concepts in that they do not model perfectly rational players, but rather seek to accurately describe the behavior of real players. These solutions concepts are of great interest for analyzing security games because they may offer more accurate predictions about the behavior of attackers.

In addition, behavioral models open new directions for reasoning about deception in security games. A solution concept that assumes perfect rationality can suggest deceptions that involve randomizing to hide information, but it cannot suggest deceptions involving false beliefs because such beliefs are inherently irrational because the definition of rationality usually includes correct Bayesian reasoning. However, behavioral models can include imperfect reasoning, including the possibility of false beliefs. Therefore, an agent with an accurate behavioral model of an opponent could develop deception strategies that exploit the imperfect reasoning of the opponent, leading to a wider range of possible deception strategies including creating false beliefs for the opponent.

8 Conclusion

We see great potential for game theory to play a role in both the theoretical foundations and practical applications of security. One area where it seems particularly relevant is modeling and analyzing strategic manipulation of information, including randomization and deception tactics. Indeed, the growing area of security games has already produced several notable successes in applying game theory to randomize security policies to improve security for physical infrastructure. There is also a small but growing number of game theoretic models that have been developed for cyber security.

We discuss a series of game models that consider optimizing the way honeypots can be used in network security. These range from the basic honeypot selection game to an extended version that allows for probing behaviors, to models that include attack graphs as a more detailed representation of attacker behavior. The models capture many of the salient features of deploying honeypots, though it is certainly possible to enrich them further. The primary limitation of these models is the scalability of the current solution algorithms, but applying techniques from other state of the art methods in computational game theory could improve the algorithms significantly. We also argue that new research on learning in dynamic game models and behavioral solution concepts that model humans will be important for improving our understanding of deception at a fundamental level, and will allow new applications of game theory for cyber security.

Acknowledgements

This research was supported by the Office of Naval Research Global (grant no. N62909-13-1-N256), and the Czech Science Foundation (grant no. P202/12/2054). Viliam Lisý is a member of the Czech Chapter of The HoneyNet Project.

References

1. B. Bosansky, C. Kiekintveld, V. Lisý, J. Cermak, and M. Pechoucek. Double-oracle algorithm for computing an exact nash equilibrium in zero-sum extensive-form games. In *Proceedings of the 12th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2013)*, 2013.
2. A. Buldas and R. Stepanenko. Upper bounds for adversaries utility in attack trees. In J. Grossklags and J. Walrand, editors, *Decision and Game Theory for Security*, volume 7638 of *Lecture Notes in Computer Science*, pages 98–117. Springer Berlin Heidelberg, 2012.
3. C. F. Camerer. *Behavioral Game Theory: Experiments in Strategic Interaction*. Princeton University Press, 2003.

4. F. Cohen. A mathematical structure of simple defensive network deception. *Computers & Security*, 19(6):520–528, 2000.
5. M. Dornseif and T. Holz. Nosebreak-attacking honeynets. In *2004 IEEE Workshop on Information Assurance and Security*, number June, pages 10–11, 2004.
6. K. Durkota and V. Lisy. Computing optimal policies for attack graphs with action failures and costs. In *STAIRS 2014: Proceedings of the Seventh Starting AI Researchers' Symposium*, page (to appear). IOS Press, 2014.
7. D. Fudenberg and D. K. Levine. *The Theory of Learning in Games*. MIT Press, 1998.
8. K. Ingols, R. Lippmann, and K. Piwowarski. Practical attack graph generation for network defense. In *Computer Security Applications Conference, 2006. ACSAC'06. 22nd Annual*, pages 121–130. IEEE, 2006.
9. M. Jain, D. Korzhuk, O. Vanek, V. Conitzer, M. Pechoucek, and M. Tambe. A double oracle algorithm for zero-sum security games on graphs. In *International Conference on Autonomous Agents and Multiagent Systems*, 2011.
10. G. S. Kc, A. D. Keromytis, and V. Prevelakis. Countering code-injection attacks with instruction-set randomization. In *Proceedings of the 10th ACM Conference on Computer and Communications Security, CCS '03*, pages 272–280, New York, NY, USA, 2003. ACM.
11. C. Kiekintveld. *Empirical Game-Theoretic Methods for Strategy Design and Analysis in Complex Games*. PhD thesis, University of Michigan, 2008.
12. C. Kiekintveld, M. Jain, J. Tsai, J. Pita, F. Ordonez, and M. Tambe. Computing optimal randomized resource allocations for massive security games. In *AAMAS-09*, 2009.
13. K. Lee. On a deception game with three boxes. *Int. Jour. of Game Theory*, 22(2):89–95, 1993.
14. V. Lisy and R. Pibil. Computing optimal attack strategies using unconstrained influence diagrams. In G. Wang, X. Zheng, M. Chau, and H. Chen, editors, *Intelligence and Security Informatics*, volume 8039 of *Lecture Notes in Computer Science*. Springer Berlin Heidelberg, 2013.
15. J. Nash. Non-cooperative games. *The Annals of Mathematics*, 54(2):286–295, 1951.
16. M. J. Osborne. *An Introduction to Game Theory*. Oxford University Press, 2004.
17. X. Ou, W. Boyer, and M. McQueen. A scalable approach to attack graph generation. In *Proceedings of the 13th ACM conference on Computer and communications security*, pages 336–345. ACM, 2006.
18. R. Pibil, V. Lisy, C. Kiekintveld, B. Bosansky, and M. Pechoucek. Game theoretic model of strategic honeypot selection in computer networks. In J. Grossklags and J. Walrand, editors, *Decision and Game Theory for Security*, volume 7638 of *Lecture Notes in Computer Science*. Springer Berlin Heidelberg, 2012.
19. J. Pita, M. Jain, C. Western, C. Portway, M. Tambe, F. Ordonez, S. Kraus, and P. Parachuri. Depoloyed ARMOR protection: The application of a game-theoretic model for security at the Los Angeles International Airport. In *AAMAS-08 (Industry Track)*, 2008.
20. J. Pita, M. Tambe, C. Kiekintveld, S. Cullen, and E. Steigerwald. GUARDS - game theoretic security allocation on a national scale. In *AAMAS-11 (Industry Track)*, 2011.
21. N. C. Rowe, E. J. Custy, and B. T. Duong. Defending Cyberspace with Fake Honeypots. *Journal of Computers*, 2(2):25–36, Apr. 2007.
22. T. Sandholm. The state of solving large incomplete-information games, and application to poker. *AI Magazine*, Special Issue on Algorithmic Game Theory, 2010.
23. T. Sandholm and S. Singh. Lossy stochastic game abstraction with bounds. In *ACM Conference on Electronic Commerce (EC)*, 2012.
24. E. Shieh, B. An, R. Yang, M. Tambe, C. Baldwin, J. Drenzo, G. Meyer, C. W. Baldwin, B. J. Maule, and G. R. Meyer. PROTECT : A Deployed Game Theoretic System to Protect the Ports of the United States. *AAMAS*, 2012.
25. Y. Shoham and K. Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009.

26. J. Spencer. A deception game. *American Mathematical Monthly*, pages 416–417, 1973.
27. L. Spitzner. *Honeypots: tracking hackers*. Addison-Wesley Professional, 2003.
28. M. Tambe. *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge University Press, 2011.
29. J. Tsai, Y. Qian, Y. Vorobeychik, C. Kiekintveld, and M. Tambe. Bayesian security games for controlling contagion. In *In Proceedings of the ASE/IEEE International Conference on Social Computing(SocialCom)*, 2013.
30. J. Tsai, S. Rathi, C. Kiekintveld, F. Ordóñez, and M. Tambe. IRIS - A tools for strategic security allocation in transportation networks. In *AAMAS-09 (Industry Track)*, 2009.